DISTRIBUTOR MANAGEMENT SYSTEM (DMS)

Prepared by:

- Rupesh Kumar Singh
- Prince Kumar
- Ragul

Reviewed by: Madan Kumar
Project Duration: 2 Months

1. Project Overview

The Distributor Management System (DMS) is a microservices-based web application designed to manage distributor operations efficiently and centrally.

Purpose:
To enable administrators to manage products, distributors, inventory, and orders while ensuring data consistency, real-time updates, and secure access.

Key Features:

- Admin and Distributor roles with secure login and logout
- Microservices-based modular architecture for scalability and flexibility
- Role-based access control and password management
- Product and distributor management
- Order creation and lifecycle tracking
- Automatic inventory updates
- Order history tracking
- Audit logging with timestamps

2. Problem Statement

Organizations that rely on distributors for product distribution often face challenges in maintaining accurate inventory levels, tracking distributor orders, and managing product information efficiently.

- Existing processes are largely manual or spread across multiple systems, leading to data inconsistencies
- Delays in order processing and fulfillment due to lack of automation
- Limited visibility into real-time inventory and distributor activities
- Increased operational errors due to fragmented data
- Lack of centralized control and secure access mechanisms

Therefore, there is a need for a centralized Distributor Management System that enables efficient management of products, inventory, and distributor orders, while providing real-time visibility, traceability, and secure operational control.

3. Objectives

- Scalability: Independent scaling of services using microservices architecture
- Data Consistency: Maintain accurate inventory and order data
- Security: Ensure authentication and role-based access
- Efficiency: Automate distributor operations
- Maintainability: Modular system design
- Reliability: Handle failures and ensure consistency

4. Functional Requirements

4.1 Authentication & User Management

- The system should allow users (Administrator and Distributor) to log in using a unique username and password.
- The system should authenticate users before granting access.
- The system should authorize actions based on user roles.
- The system should allow users to log out securely.
- The system should allow users to change their password.
- The system should prevent unauthorized access to restricted modules.

4.2 Product Management

- The administrator should be able to add new product details
- The administrator should be able to update product information
- The administrator should be able to delete product records
- The administrator should be able to view all products
- The system should maintain product details such as name, price, and stock

4.3 Distributor Management

- The administrator should be able to add new distributors
- The administrator should be able to update distributor details
- The administrator should be able to view distributor information
- The system should store distributor details such as contact and address

4.4 Order Management

- The distributor should be able to place orders

- The system should store order details in the database
- The administrator should be able to view all orders
- The administrator should be able to update order status (Created → Approved → Dispatched → Delivered)
- The distributor should be able to view order status
- The distributor should be able to view order history

4.5 Inventory Management

- The system should maintain stock levels for products
- The system should automatically update stock when an order is placed
- The system should validate stock availability before order placement
- The system should prevent orders if stock is insufficient
- The administrator should be able to monitor inventory levels
- The system should restore inventory if order processing fails

4.6 Audit Logging

- The system should log important actions such as login, order creation, and updates
- The system should maintain timestamps for system activities
- The administrator should be able to view audit logs

4.7 Exit Handling

- The system should allow users to log out securely
- The system should invalidate sessions after logout

4.8 System Constraints

- The system should handle failures gracefully
- The system should maintain data consistency
- The system should prevent invalid operations

4.9 Dashboard & Reporting

• The system should provide a dashboard for administrators

• The dashboard should display order summary and inventory status

• The system should provide basic reports for monitoring activities

5. Non-Functional Requirements

- Scalability: Support multiple users and requests
- Availability: Minimal downtime
- Performance: Fast API response
- Security:
    - Secure authentication and authorization
    - Role-based access control
    - Password management and session handling
- Reliability: No data inconsistency during operations

6. Scope

In-Scope:

- Product management
- Distributor management

- Order processing
- Inventory tracking
- REST API-based communication

Out-of-Scope:

- Payment systems
- Delivery tracking
- External integrations

7. Architecture

• **Architecture Type:** Microservices Architecture

Services:

• User Service (authentication and user management)

• Product Service (product management)

• Distributor Service (distributor management)

• Order Service (order processing and lifecycle)

• Inventory Service (stock validation and updates)

• Audit Service (logging and monitoring)

API Gateway:

• Handles incoming client requests and routes them to appropriate services

Database:

• Each service maintains its own MySQL database (Database per service pattern)

Communication:

• REST APIs between services

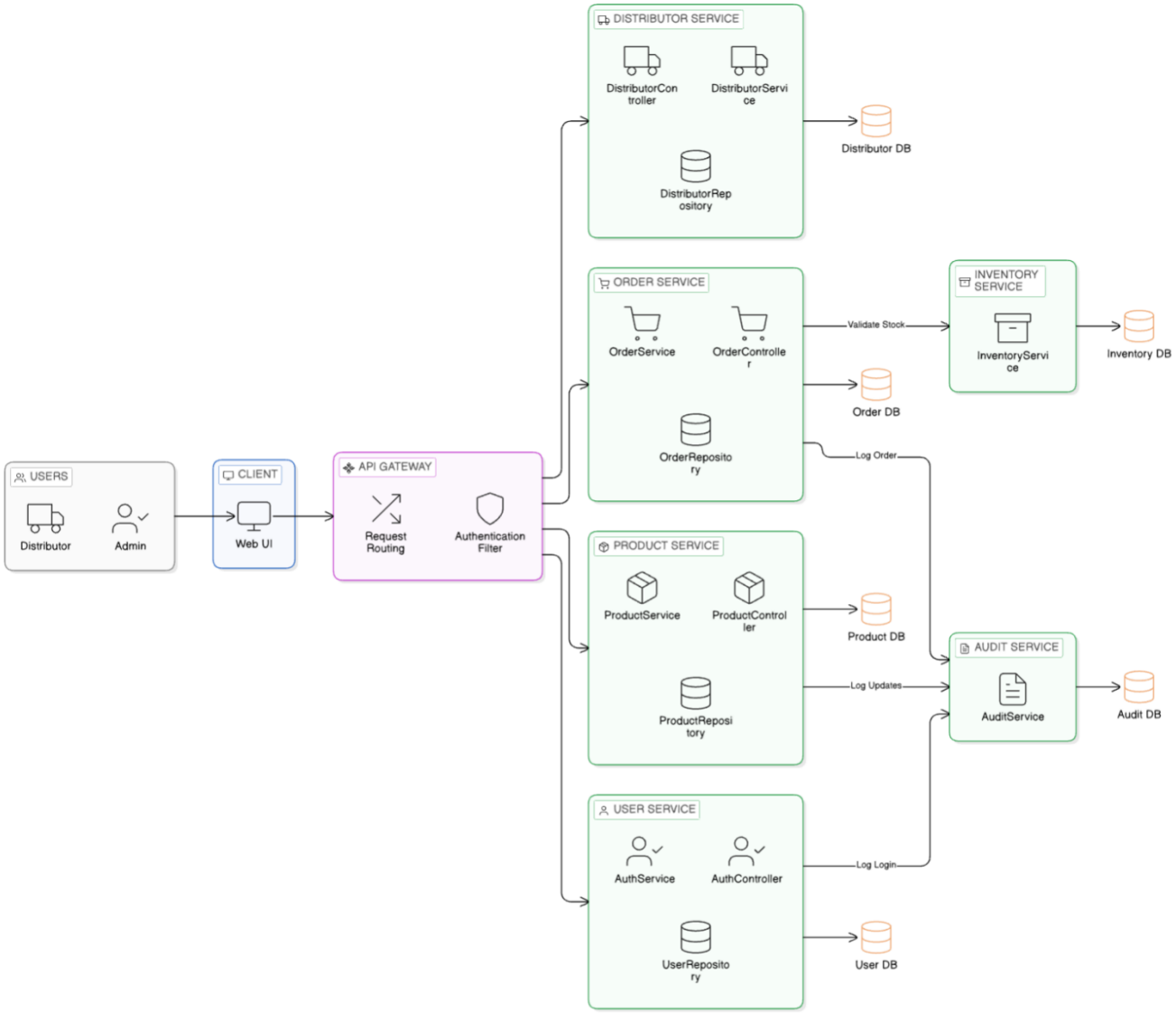
• API Gateway used for external communication

Data Handling:

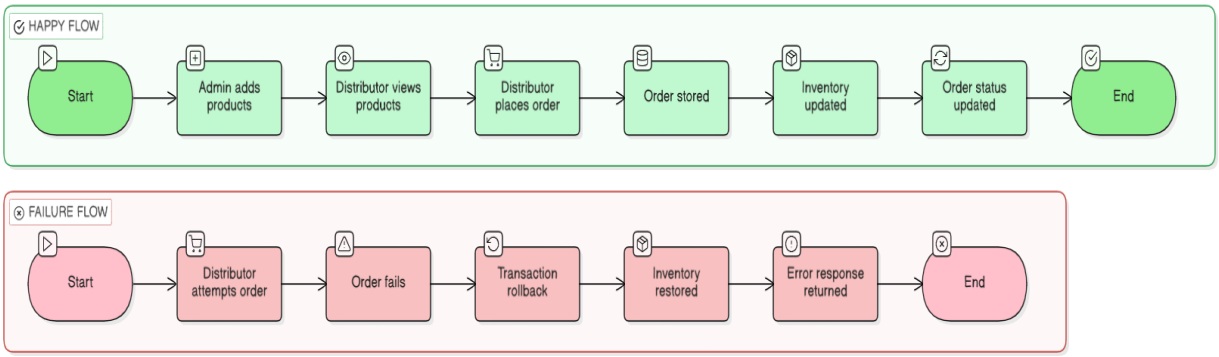
• Transactions handled within each service using Spring Boot & JPA

• Inventory Service ensures stock validation and updates

Ensures data consistency across services using service coordination



8. Flow Diagrams



9. ER Diagram

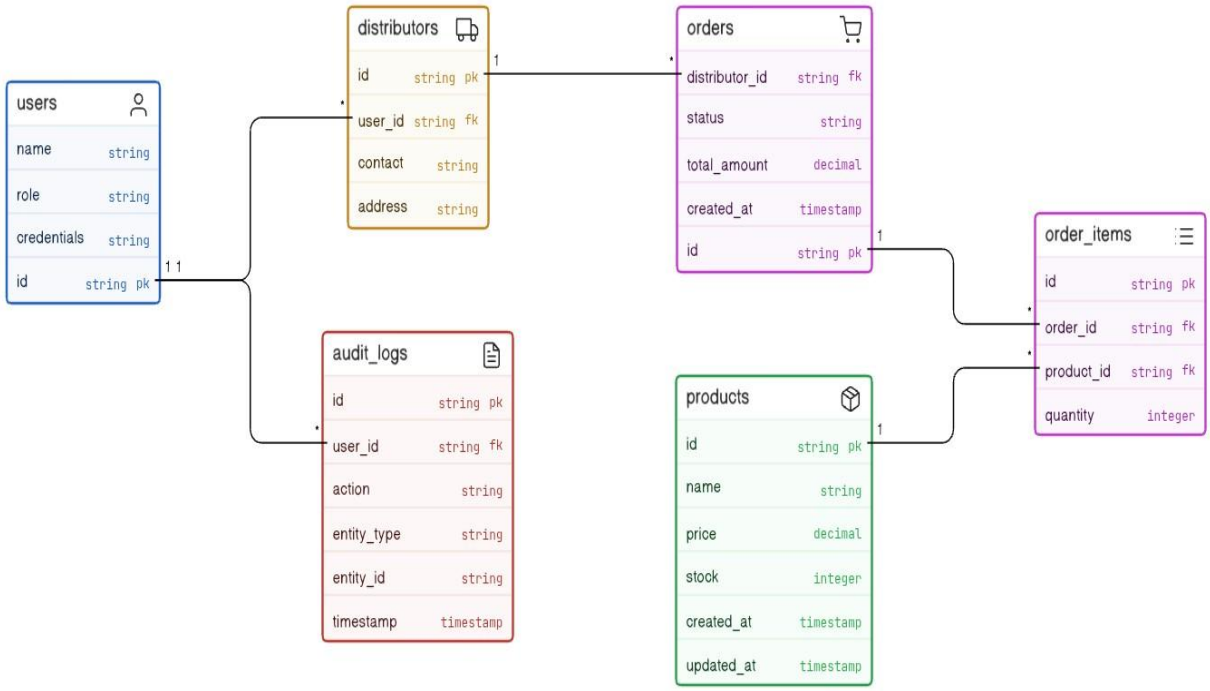
- Each service maintains its own database schema (loosely coupled design)

Key Entities:

- User: id, name, role, credentials
- Product: id, name, price, stock
- Distributor: id, details
- Order: id, distributor_id, status
- Order Item: id, product_id, quantity
- AuditLog: id, user_id, action, timestamp

Relationships:

- One User → Many Orders
- One Order → Many Order Items
- One Product → Many Order Items



10. Technology Stack

- Backend: Spring Boot, Hibernate, Spring Data JPA
- Architecture: Microservices-based Spring Boot services
- Database: MySQL
- Security: Spring Security (authentication & authorization)
- Communication: REST APIs
- Testing: Postman
- Build Tool: Maven
-

11. Data Storage Strategy

• Database per service approach (loose coupling)

• Each microservice manages its own MySQL database

• Transactions handled within individual services

• Maintain consistency using service-level coordination

12. Security

- Role-based access control (Admin / Distributor)
- Secure login and logout
- Password management (change password)
- Session management
- Input validation
- Protection against unauthorized access
- Password encryption for secure storage
- Centralized authentication via User Service
- Secure communication between services

13. Performance

- Efficient database queries
- Services can be scaled independently
- Proper indexing (product_id, order_id)
- Fast API response time
- Optimized backend logic

14. Testing Strategy

- Unit Testing (JUnit)
- API Testing (Postman)
- Integration Testing
- Testing failure scenarios

- Test login/logout and role-based access
- Test inventory validation (insufficient stock cases)
- Test transactional behavior (rollback on failure)

15. Future Enhancements

• Notification system for order updates

• Advanced analytics and reporting features

• Cloud deployment and scalability improvements

16. Timeline

Week	Module / Focus	Tasks	Deliverables
Week 0	Project Setup	- Finalize requirements - Setup Git repo - Install MySQL, IDE, tools - Design DB schema	Project setup ready, DB design
Week 1	Authentication Module	Implement login & logout- Password handling / change password- Role-based access control	Working Authentication APIs
Week 2	Product Management	- Add / update / delete products- View product list- Connect with DB	Product module completed
Week 3	Distributor Management	- Add distributor- Update distributor details- View distributor list	Distributor module completed
Week 4	Order Management (Core)	- Create order API- Order items handling- Store order data	Order creation working
Week 5	Inventory Integration	- Auto update stock on order- Validate stock availability- Handle failures	Inventory logic integrated
Week 6	Order Lifecycle	- Update order status- Implement flow (Created → Delivered)- Order tracking	Full order workflow ready
Week 7	Testing & Security	- API testing (Postman)- Fix bugs- Add security (validation, roles)- Integration testing	Tested & stable system
Week 8	Finalization	- Performance check- Documentation- Prepare diagrams- Demo preparation	Final project ready

17. Conclusion

The Distributor Management System provides a centralized, scalable, and secure solution for managing distributor operations. It ensures efficient order processing, accurate inventory tracking, and improved visibility, making it a reliable system for real-world applications.